Drone Audio Detection with Various Models

M. Xavier Adams and Calvin Hinkle

Problem Statement

- Drone technology is advancing rapidly
- More common in everyday life
- Integral part of modern military actions
- Can drones be detected by ML using only the sound they make?

Data

- Labeled audio files sourced from Kaggle[1]
- Labeled as either “Drone” or “Unknown”


- Provided as .wav files
- Consistent sample rate
- ~11% “Drone”
- ~1 second clips

[1] https://www.kaggle.com/datasets/amineipad/drone-sound-audio-detection

Prior work

“Drone Audio recognition based on Machine Learning Techniques”

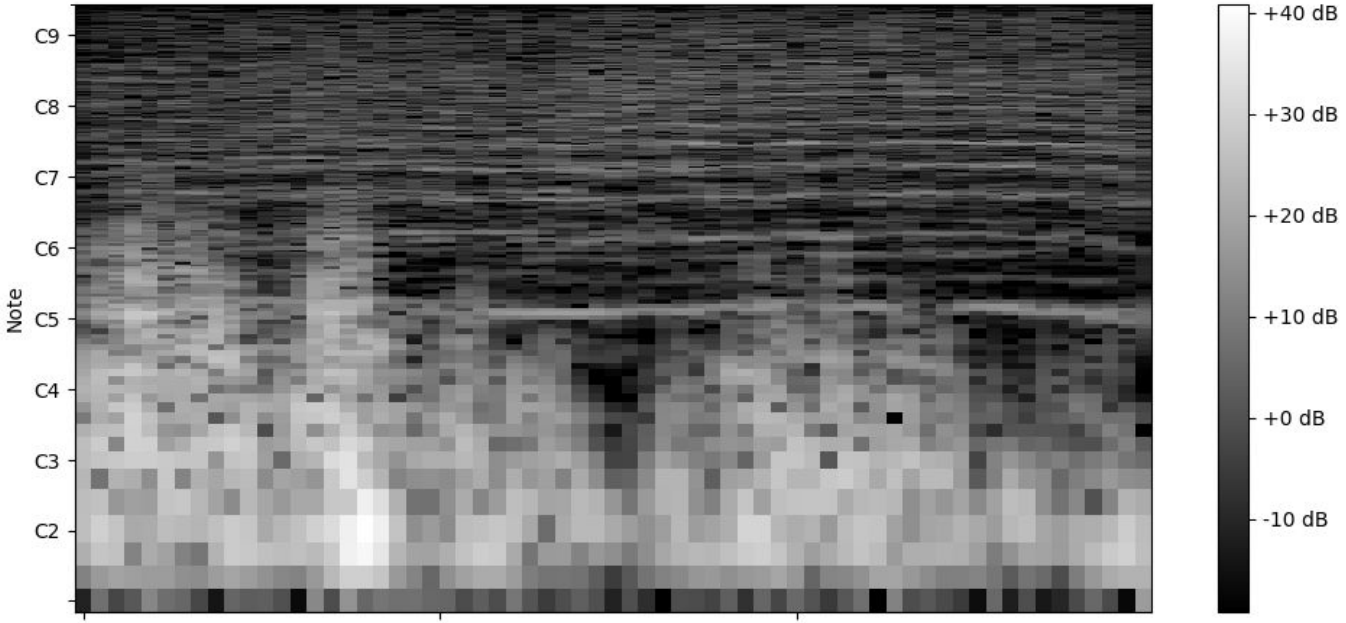
- What ML models perform and scale well?
- Clean, well-labeled data
- 2020 in Italy

Feature Extraction

- Librosa: Python audio processing package
  - Spectrogram
  - Spectral Features
  - MFCC
- FFT

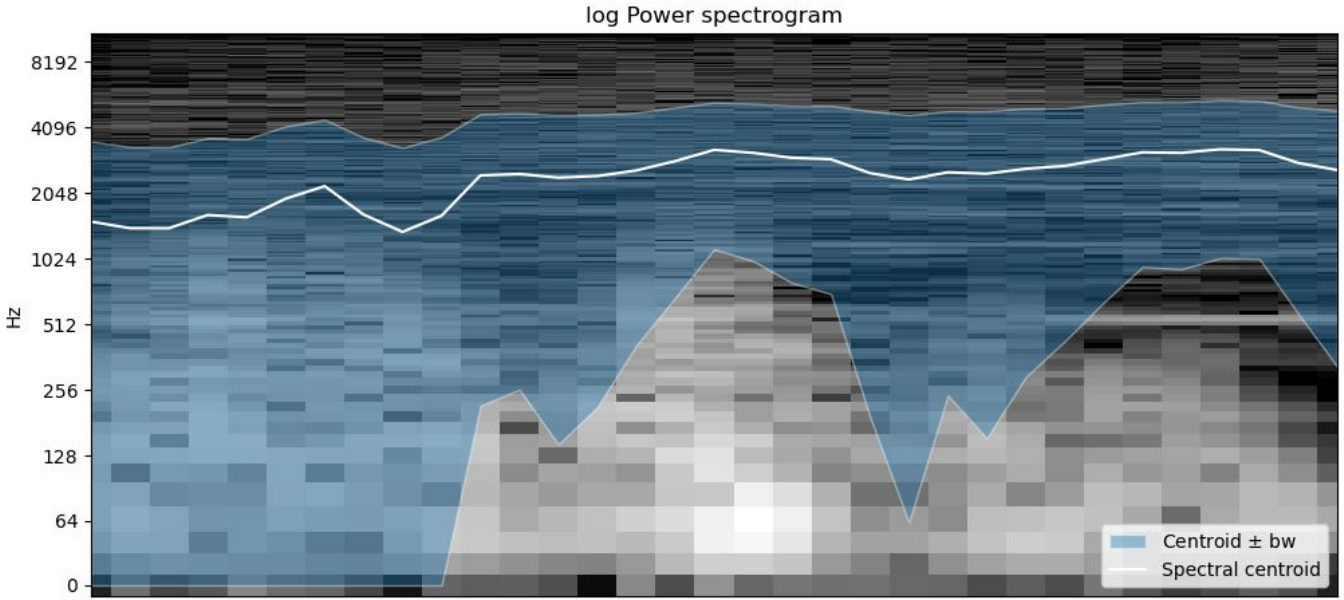
Spectrogram

Spectrum of frequencies in a signal



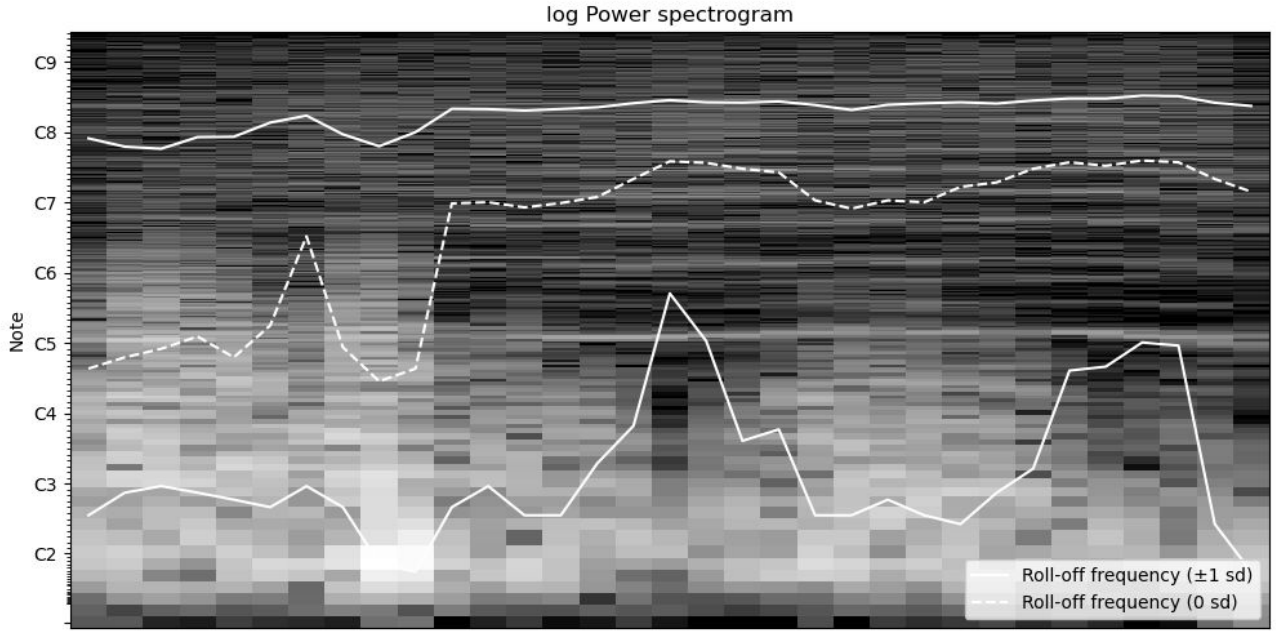
Spectral Features

Centroid (weighted avg of frequencies) **, bandwidth** (range of frequencies)



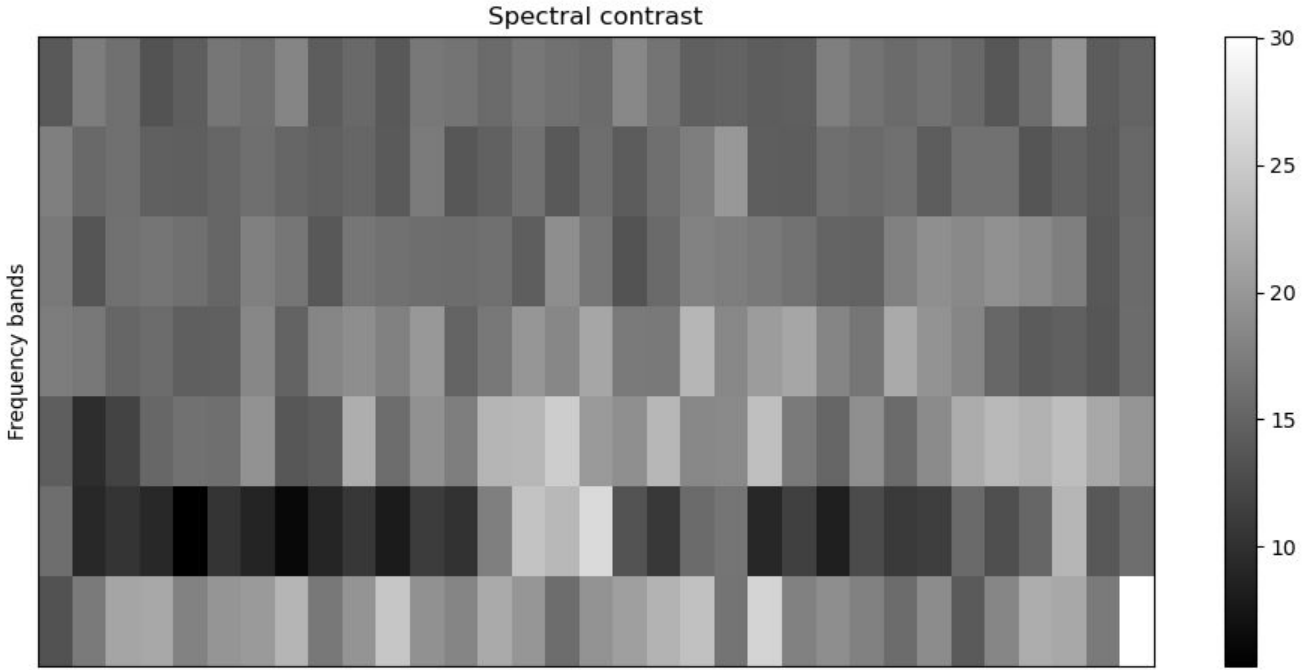
Spectral Features

Rolloff (percentiles of energy)



Spectral Features

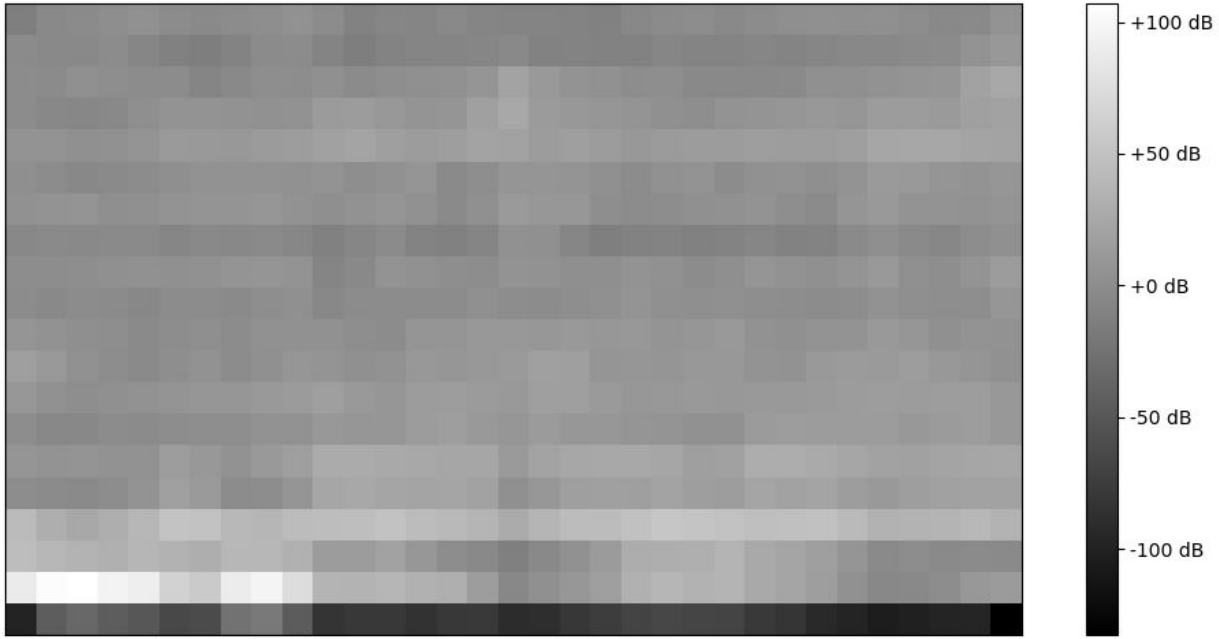
Contrast (peaks-valleys → **narrow vs broad** bands of energy**)**



MFCC

Mel Frequency Cepstral Coefficients (MFCC)

Describe shape of spectral envelope (timbre, scaled how humans hear)



Fast Fourier Transform

- Transform data from time to frequency space
- Nyquist–Shannon sampling theorem
  - Up to samplerate / 2 = 8000 Hz
- Length of clip determines spacing
  - Interpolate for consistency
  - Tighter spacing towards lower end of spectrum

Solution(s)

```
sv_model = GridSearchCV(
    estimator=LinearSVC(),
    param_grid=SV_PARAM_GRID,
    scoring="f1",
    cv=CROSS_VAL,
    n_jobs=NUM_WORKERS,
)
```

```
lr_model = GridSearchCV(
    estimator=LogisticRegression(),
    param_grid=LR_PARAM_GRID,
    scoring="f1",
    cv=CROSS_VAL,
    n_jobs=NUM_WORKERS,
)
```

```
svm_model = Pipeline([
    ("scaler", StandardScaler()),

    ("classifier", LinearSVC(
        class_weight="balanced",
        max_iter=50000,
        dual=False,
        random_state=RANDOM_SEED) )
])
```

```
log_model = Pipeline([("scaler", StandardScaler()),

                      ("classifier", LogisticRegression(
                          max_iter=5000,
                          class_weight="balanced",
                          random_state=RANDOM_SEED)
                      )])
```

Solution(s)

- Neural Network
  - Kept it simple
- Random Forest

```python
class NeuralNet(torch.nn.Module):
    def __init__(self, input_dim, hidden_sizes):
        super().__init__()
        layers = []
        prev_dim = input_dim
        for hidden_dim in hidden_sizes:
            layers.append(torch.nn.Linear(prev_dim, hidden_dim))
            layers.append(torch.nn.ReLU())
            prev_dim = hidden_dim
        layers.append(torch.nn.Linear(prev_dim, 2))
        self.network = torch.nn.Sequential(*layers)

    def forward(self, x):
        return self.network(x)
```

```python
class_counts = torch.bincount(train_labels, minlength=2)
loss_class_weights = len(train_labels) / (len(class_counts) * class_counts.float())
criterion = torch.nn.CrossEntropyLoss(weight=loss_class_weights)
optimizer = torch.optim.Adam(model.parameters(), lr=float(params["learning_rate"]))
```

```python
rf_model = GridSearchCV(
    estimator=RandomForestClassifier(),
    param_grid=RF_PARAM_GRID,
    scoring="f1",
    cv=CROSS_VAL,
    n_jobs=NUM_WORKERS,
)
```

Solution(s)

```
LR_PARAM_GRID = [
    {
        "C": np.logspace(-6.0, 6.0, num=7, base=2.0),
        "l1_ratio": [0.0],
        "class_weight": ["balanced"],
        "random_state": RANDOM_SEEDS,
        "solver": ["lbfgs", "sag"],
    },
    {
        "C": np.logspace(-6.0, 6.0, num=7, base=2.0),
        "l1_ratio": [0.0, 1.0],
        "class_weight": ["balanced"],
        "random_state": RANDOM_SEEDS,
        "solver": ["liblinear"],
    },
    {
        "C": np.logspace(-6.0, 6.0, num=7, base=2.0),
        "l1_ratio": np.linspace(0.0, 1.0, 11),
        "class_weight": ["balanced"],
        "random_state": RANDOM_SEEDS,
        "solver": ["saga"],
    },
]
```

```
NN_PARAM_GRID = [
    {
        "learning_rate": np.logspace(-17, -11, num=4, base=2.0),
        "batch_size": np.logspace(6.0, 10.0, num=3, base=2.0).astype(int),
        "hidden_sizes": [
            (
                NUM_FEATURES // 2,
                NUM_FEATURES // 4,
                NUM_FEATURES // 8,
                NUM_FEATURES // 16,
            ),
        ],
        "max_epochs": [2**8],
        "train_converge_window": [2**5],
        "eval_step": [2**2],
        "random_state": RANDOM_SEEDS,
    },
]
```

```
RF_PARAM_GRID = [
    {
        "n_estimators": np.logspace(6.0, 9.0, num=4, base=2.0).astype(int),
        "criterion": ["gini"],
        "max_depth": [None],
        "min_samples_split": np.logspace(1.0, 3.0, num=3, base=2.0).astype(int),
        "min_samples_leaf": np.logspace(0.0, 1.0, num=2, base=2.0).astype(int),
        "max_features": ["sqrt", "log2"],
        "random_state": RANDOM_SEEDS,
        "class_weight": ["balanced"],
    },
]
```

```
SV_PARAM_GRID = [
    {
        "penalty": ["l2"],
        "loss": ["squared_hinge", "hinge"],
        "C": np.logspace(-6.0, 6.0, num=7, base=2.0),
        "class_weight": ["balanced"],
        "random_state": RANDOM_SEEDS,
    },
    {
        "penalty": ["l1"],
        "loss": ["squared_hinge"],
        "C": np.logspace(-6.0, 6.0, num=7, base=2.0),
        "class_weight": ["balanced"],
        "random_state": RANDOM_SEEDS,
    },
]
```

Assumptions

Dataset is **representative** of the real world

- Reshaped new data will work → new data is the same quality and type as train
- Kaggle data is labeled correctly (Drone→drone, unknown→unknown)
  - Data is clean and separable
- “Unknown” covers test-space well
- Short clips work well for drones
- Drones are about as common as in testing (1:10 clips)

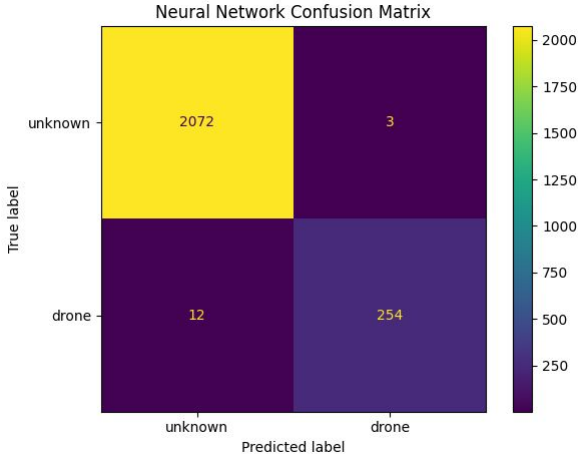
Features are **representative** of drone signatures

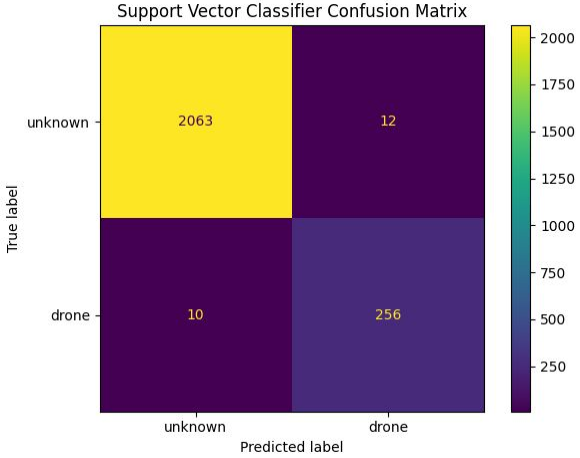
Constraints and Limitations

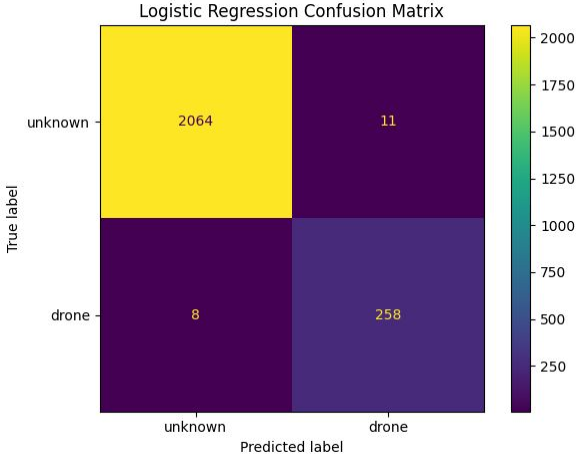
- Compute
  - All models were trained and evaluated on our personal computers
- Time
  - More thorough tuning and with more models
- Nyquist limit → lack of high frequency

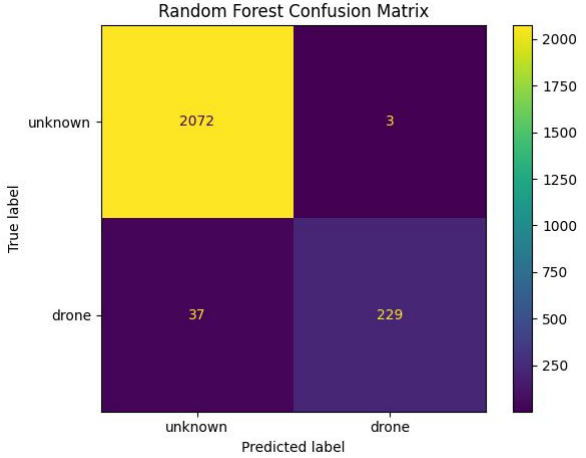
Results

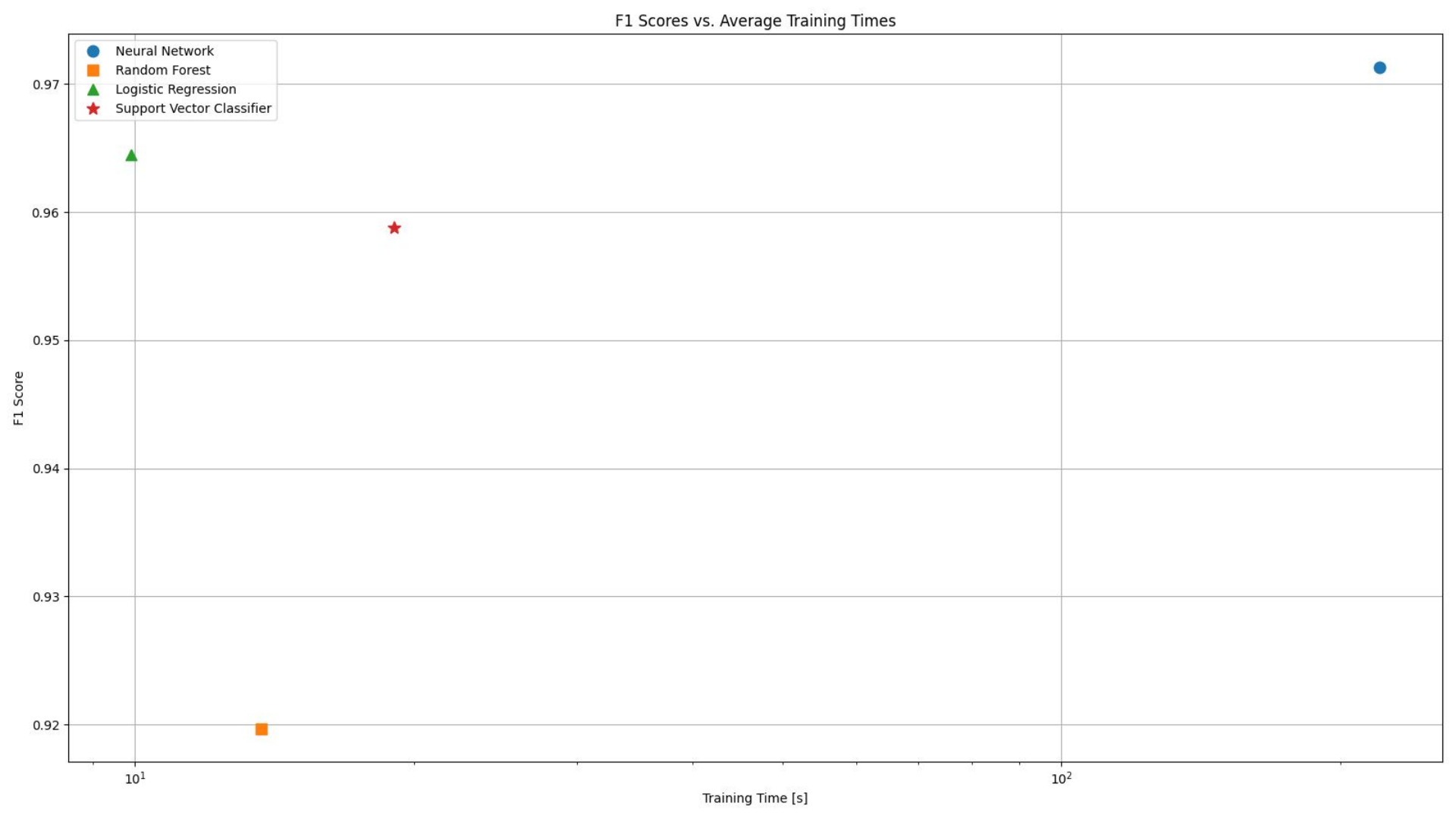
- Neural Network performed the best
  - F1 score of ~0.97










F1 Scores vs. Average Training Times
Neural Network
Random Forest
Logistic Regression
Support Vector Classifier
0.97
0.96
0.95
0.94
0.93
0.92
F1 Score
10^1
10^2
Training Time [s]

Compared to paper

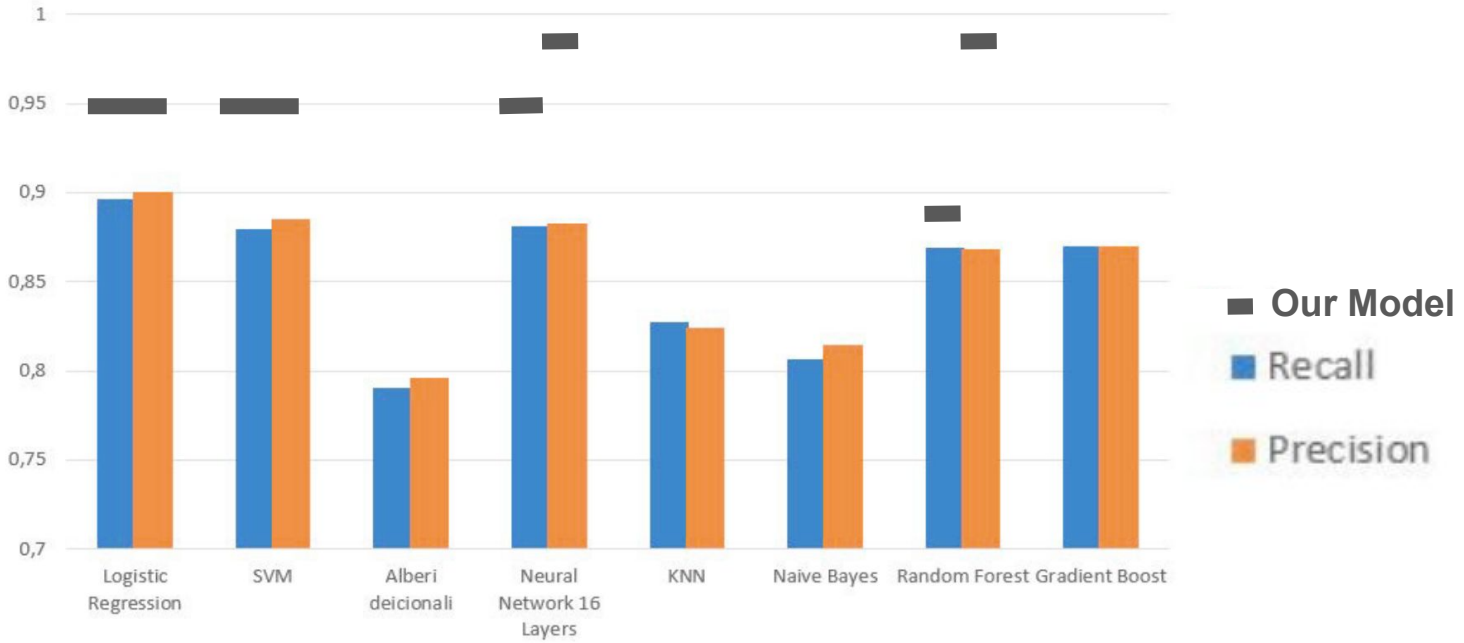


Fig. 8. *Summary of Precision and Recall average values for the eight models used*

Demo!

Let’s use our model on some clips not in our data set and see how it performs

Cow

Lawn Mower

Drone

Results

Demo!

Let’s use our model on some clips not in our data set and see how it performs

Cow

Lawn Mower

Drone

Results

```
File: cow.wav
LogReg prediction: 1 = other
LogReg probability of drone: 0.0000%
SVM prediction: 1 = other
SVM score: 7.82
```

Demo!

Let’s use our model on some clips not in our data set and see how it performs

Cow **Lawn Mower** **Drone**



Results

```
File: cow.wav
LogReg prediction: 1 = other
LogReg probability of drone: 0.0000%
SVM prediction: 1 = other
SVM score: 7.82
```

```
File: lawn_mower.wav
LogReg prediction: 1 = other
LogReg probability of drone: 18.2647%
SVM prediction: 1 = other
SVM score: 1.55
```

Demo!

Let’s use our model on some clips not in our data set and see how it performs

Cow

Lawn Mower

Drone

Results

```
File: cow.wav
LogReg prediction: 1 = other
LogReg probability of drone: 0.0000%
SVM prediction: 1 = other
SVM score: 7.82
```

```
File: drone.wav
LogReg prediction: 0 = Drone
LogReg probability of drone: 97.2776%
SVM prediction: 0 = Drone
SVM score: -0.75
```

```
File: lawn_mower.wav
LogReg prediction: 1 = other
LogReg probability of drone: 18.2647%
SVM prediction: 1 = other
SVM score: 1.55
```

Summary

- Drone audio detection in a relevant problem
- Spectrogram and frequency analysis feature engineering
- Neural Networks perform best, training takes the longest
    - Logistic Regression if training time is constrained

Questions?

Works Cited

- “Drone Audio recognition based on Machine Learning Techniques.” Lecava et al. KES 2022 10.1016/j.procs.2022.09.140
- https://www.kaggle.com/datasets/amineipad/drone-sound-audio-detection